

Tokenizer Quirks

Small details in tokenization, big effects on the model · video 2:40

This lesson answers one question: how do small details of tokenization show up as strange model behaviour? Four quirks, each tested for real: the same word as **many different tokens**; **numbers** cut into irregular pieces; a **trailing space** that derails a prediction; and **glitch tokens** that were in the vocabulary but almost never in the training text.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The same word, many tokens	▶ 0:08-0:38	2	☐
2. Numbers	▶ 0:38-1:01	2	☐
3. The trailing space	▶ 1:01-1:24	2	☐
4. Glitch tokens	▶ 1:24-2:37	2	☐

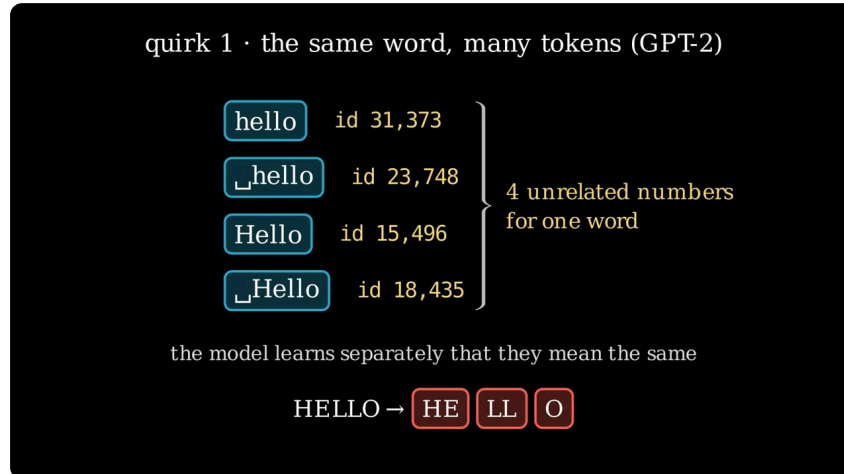
Before you start: Deep Dive episodes 1-2 (BPE, bytes and Unicode) and How LLMs Work episode 3 (embeddings). Code: `code/d03_tokenizer_quirks` (downloads Qwen2.5-0.5B and GPT-2, about 1.5 GB).

CONCEPT 1 OF 4

The same word, many tokens

▶ Video 0:08–0:38

In GPT-2, *hello*, *hello* (with a space in front), *Hello* and *Hello* are four different tokens with unrelated numbers: 31,373, 23,748, 15,496 and 18,435. The model has to learn, separately, that they mean the same thing. And *HELLO* is not even one token: it is three pieces, HE | LL | O.



▶ 0:37 hello, hello, Hello and Hello: four GPT-2 tokens with unrelated ids; HELLO is three pieces.

KEY IDEA

Token ids carry no notion of “the same word”: every variant is learned from its own examples.

Check 1: The same word, many tokens

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

GPT-2 ids: “ World” 2159, “World” 10603, “ world” 995, “world” 6894. How many different tokens is that for one word?

1.2 TRUE OR FALSE

“Token ids that are numerically close (like 995 and 996) have similar meanings.”

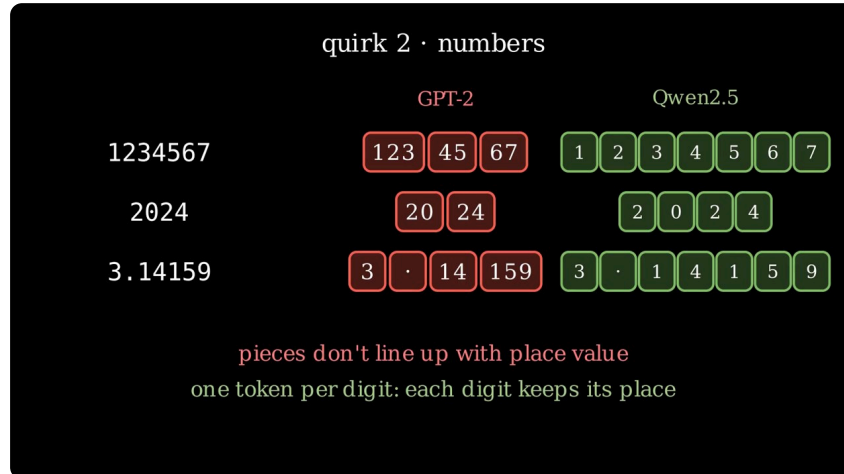
True ☐ False ☐

CONCEPT 2 OF 4

Numbers

▶ Video 0:38–1:01

GPT-2 cuts 1234567 into **123 | 45 | 67**, 2024 into **20 | 24** and 3.14159 into 3 | . | 14 | 159. The pieces do not line up with place value, which makes arithmetic harder to learn. **Qwen2.5** splits every number into **single digits**, so each digit keeps its place; many newer models do the same.



▶ 1:00 GPT-2 cuts numbers into irregular chunks; Qwen2.5 uses one token per digit.

KEY IDEA

Consistent digit tokens let the model learn arithmetic as operations on digits, the way we do on paper.

Check 2: Numbers

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 SHORT ANSWER

GPT-2 splits 12345 into 123 | 45, and 1000000 into 1 | 000000. Why does this make adding numbers hard to learn?

.....

2.2 CALCULATION

How many tokens is 1000000 for Qwen2.5 (one token per digit)?

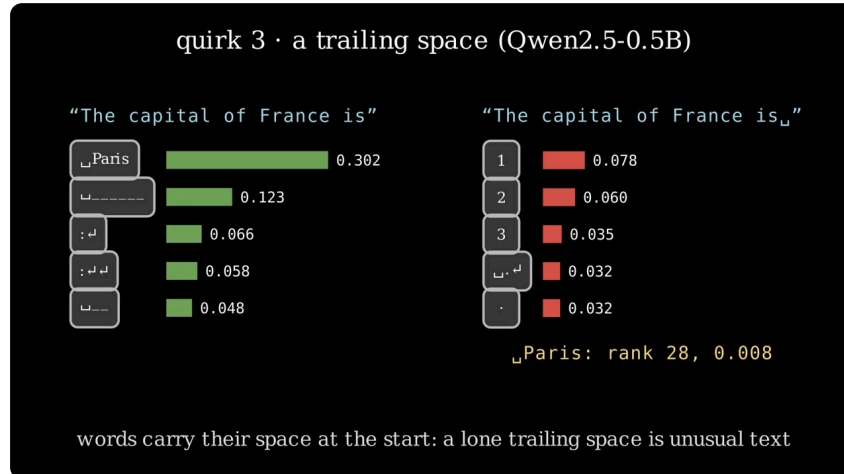
.....

CONCEPT 3 OF 4

The trailing space

▶ Video 1:01-1:24

Given “The capital of France is”, Qwen2.5-0.5B's top guess is “ **Paris**”, **0.302**. Add one space at the end, and “ Paris” drops to **rank 28** (0.008); the top guess becomes the digit 1. Words carry their space at the start, so a prompt ending in a lone space is unusual text, and the model is thrown off.



▶ 1:23 One trailing space: “ Paris” falls from 0.302 to rank 28 (0.008).

KEY IDEA

Prompts should end where a natural token boundary is: never with a trailing space.

Check 3: The trailing space

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

For “The sky is”, Qwen2.5-0.5B's top guesses are “ a” (0.129) and “ blue” (0.107). With a trailing space, “The sky is ”, what happens?

- A. “ blue” jumps to the top
- B. Nothing changes
- C. Digits take over: “1” (0.31), “3”, “5”
- D. The model refuses

3.2 SHORT ANSWER

Why does “ Paris” (with a leading space) become unlikely after a trailing space?

.....

CONCEPT 4 OF 4

Glitch tokens

▶ Video 1:24-2:37

GPT-2's vocabulary contains strange single tokens like “ `SolidGoldMagikarp`” (id 43,453), a username from a Reddit forum. The vocabulary was built from one collection of text, and the model was trained on another, where these strings almost never appear. In early models of that family, asking about them famously produced bizarre answers.

quirk 4 · glitch tokens

`SolidGoldMagikarp` one token, id 43,453
 `TheNitromeFan` one token, id 42,090
 `davidjl` one token, id 23,282
(SolidGoldMagikarp: a username from a Reddit forum)

vocabulary built from text A
model trained on text B, where they almost never appear

asked about them, early models of that family produced bizarre answers

▶ 1:46 Glitch tokens: single GPT-2 tokens whose strings almost never appear in the training text.

never seen in training → almost no updates → a generic embedding

closest to the average (GPT2):

- `externalToEVA`
- control chars
- broken bytes (🔪)
- quickShip

average embedding

`SolidGoldMagikarp`: rank 14,357 of 50,257, not flagged

▶ 2:11 Embeddings closest to the average: control characters, broken bytes, externalToEVA, quickShip; SolidGoldMagikarp is not flagged.

A token that is never seen in training gets almost no updates, so its embedding stays generic. A rough test, the embeddings closest to the average, finds control characters, broken byte pieces and odd strings like “externalToEVA” and “quickShip” (themselves known glitch tokens), but “ `SolidGoldMagikarp`” ranks 14,357 of 50,257: not flagged. No single check finds them all. Lessons: no trailing spaces, consistent spelling and capitalization, digit-splitting tokenizers for math, and look at how your own data is tokenized.

KEY IDEA

Every token needs training data. Vocabulary built from different text than the model was trained on leaves under-trained tokens.

Check 4: Glitch tokens

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 SHORT ANSWER

Why does a token that never appears in the training text keep a “generic” embedding?

4.2 HANDS-ON CODE

Try it yourself. Run this with the GPT-2 tokenizer from `quirks.py`. How many tokens is each string, and which ones are single tokens?

```
for w in [" SolidGoldMagikarp", " TheNitromeFan", " davidjl", " cat", " HELLO"]:  
    print(repr(w), len(gpt2(w)["input_ids"]))
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The same word, many tokens · rewatch ▶ 0:08-0:38

1.1 4. Case and the leading space each produce a separate vocabulary entry.

1.2 **False.** Ids are just positions in the vocabulary, in merge order; meaning lives in the learned embeddings.

2. Numbers · rewatch ▶ 0:38-1:01

2.1 **The same digit position falls in different tokens depending on the number, so the model cannot line up ones, tens, hundreds consistently.** Column addition needs aligned digits; irregular chunks hide the columns.

2.2 7. Seven digits, seven tokens (GPT-2: 2).

3. The trailing space · rewatch ▶ 1:01-1:24

3.1 **C.** Checked by running the model: the trailing space makes the text unusual, and digits become the likeliest continuation.

3.2 **The space is already in the text, so a second space token is unlikely; and “Paris” without a space is a rare token in that position.** The model learned that a word's space comes attached to the word, not before it.

4. Glitch tokens · rewatch ▶ 1:24-2:37

4.1 **Its embedding row only receives gradient updates when the token appears in a training example; if it never appears, the row stays close to where it started.** Embeddings are learned from use (How LLMs Work, episodes 3 and 11).

4.2 **“_SolidGoldMagikarp”, “_TheNitromeFan”, “_davidjl” and “_cat” are 1 token each; “_HELLO” is 2 (_HELL | O).** Rare usernames got their own tokens because they were frequent in the text the vocabulary was built from.